

RTOS
Les systèmes d'exploitation temps réel

SMUEL301

Cours de M2 EsTel - UCA

François Verdier

LEAT - UMR CNRS 7248

Préambule

Les cours, les TD et les TP que je vous donne à l'Université Côte d'Azur sont tous l'illustration de ce que je pense être **un apprentissage**, pour vous, de certaines technologies, techniques ou compétences liées à l'électronique, l'informatique industrielle ou l'informatique générale.

Ces cours, TD ou TP **ne sont pas**, pour moi, juste un moyen **de vous donner des notes !**

En conséquence, **je ne peux pas accepter** que des étudiants me rendent **des rapports de TP** fait à l'aide d'**une IA générative** quelconque. Dans la mesure où vous n'êtes pas encore capable de comprendre le contenu de mes cours, je ne peux pas accepter un rapport de TP auquel vous ne comprendriez rien du tout lors d'une lecture ultérieure.

En conclusion, je ne vous demanderai plus de rapport de TP. Je vous interrogerai sur ce que vous aurez fait en TP et je vous donnerai la note à l'issue de ça ! Il vous restera, pour vous, à apprendre à écrire correctement et à faire des rapports de ce que vous ferez dans l'industrie.

Plan du cours

1. Introduction
2. Architecture d'un RTOS
3. Les services minimum
4. L'ordonnancement
5. Un petit aperçu des RTOS
6. Le RTOS Zéphyr
7. La carte de développement EDU32F103 pour Zéphyr

Chapitre 1

Introduction



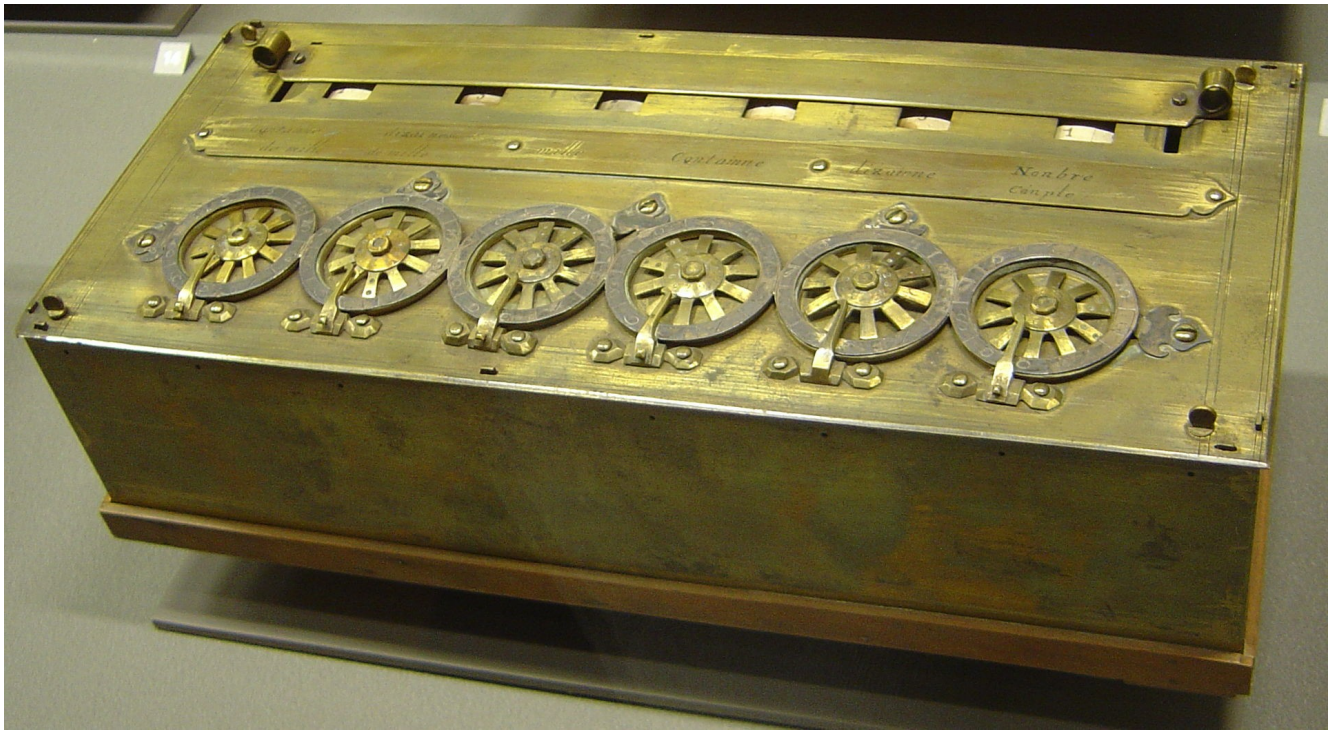
1.1 Histoire de l’informatique, rappels

1. Point de vue langages

Génération 0 : antérieure à 1957

- programmation directe des mémoires en langage machine par des interrupteurs à deux positions (clefs)...

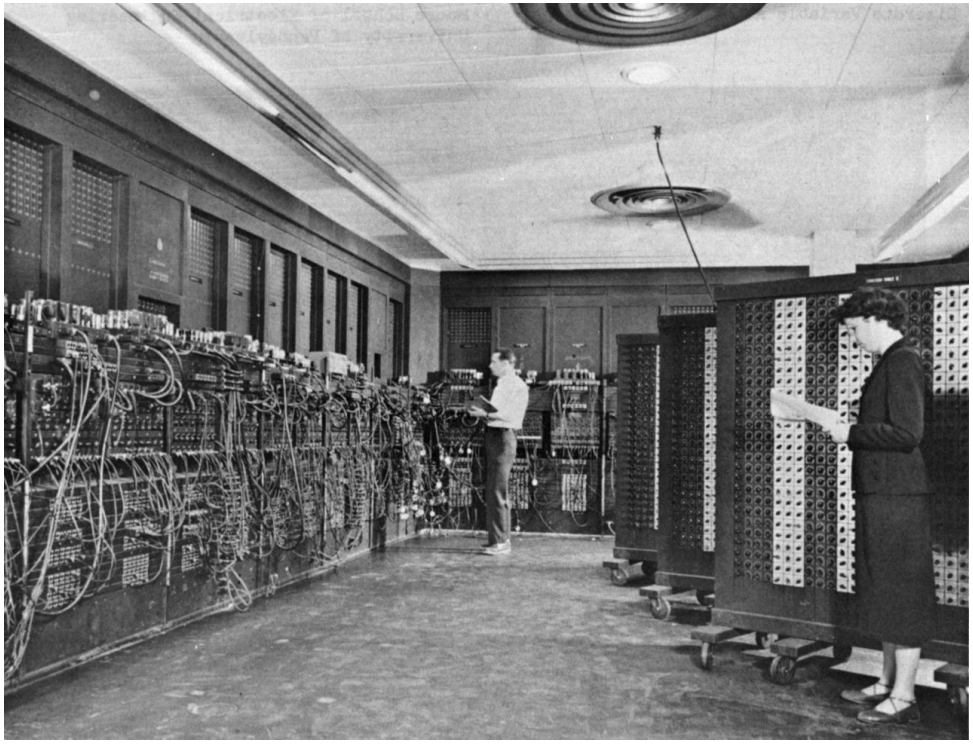
La machine à calculer de Pascal (La Pascaline) 1652



1. Point de vue langages

Génération 0 : antérieure à 1957

- programmation directe des mémoires en langage machine par des interrupteurs à deux positions (clefs)…

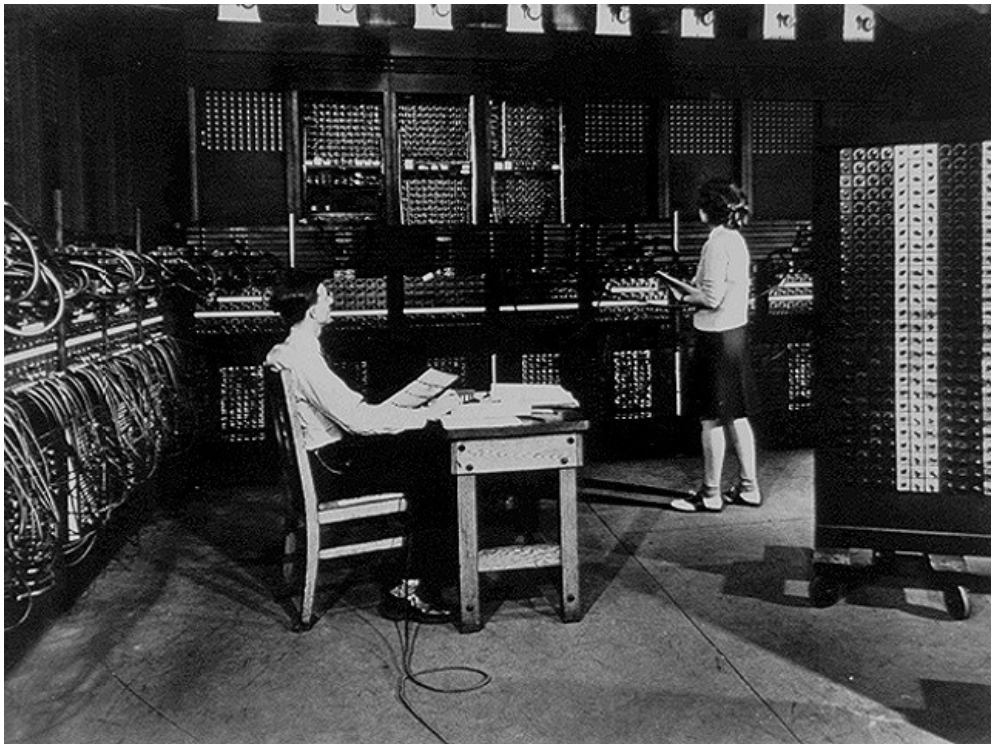


L'ordinateur ENIAC conçu par
P. Eckert et J.W. Mauchly
Electronic Numerical Integrator and Computer en 1945

1. Point de vue langages

Génération 0 : antérieure à 1957

- programmation directe des mémoires en langage machine par des interrupteurs à deux positions (clefs)…



L'ordinateur ENIAC conçu par
P. Eckert et J.W. Mauchly
Electronic Numerical Integrator and Computer en 1945

1. Point de vue langages

Génération 0 : antérieure à 1957

- programmation directe des mémoires en langage machine par des interrupteurs à deux positions (clefs)…

1ère génération : 1957

- premiers assembleurs

- premiers langages (très peu expressifs) : FORTRAN I

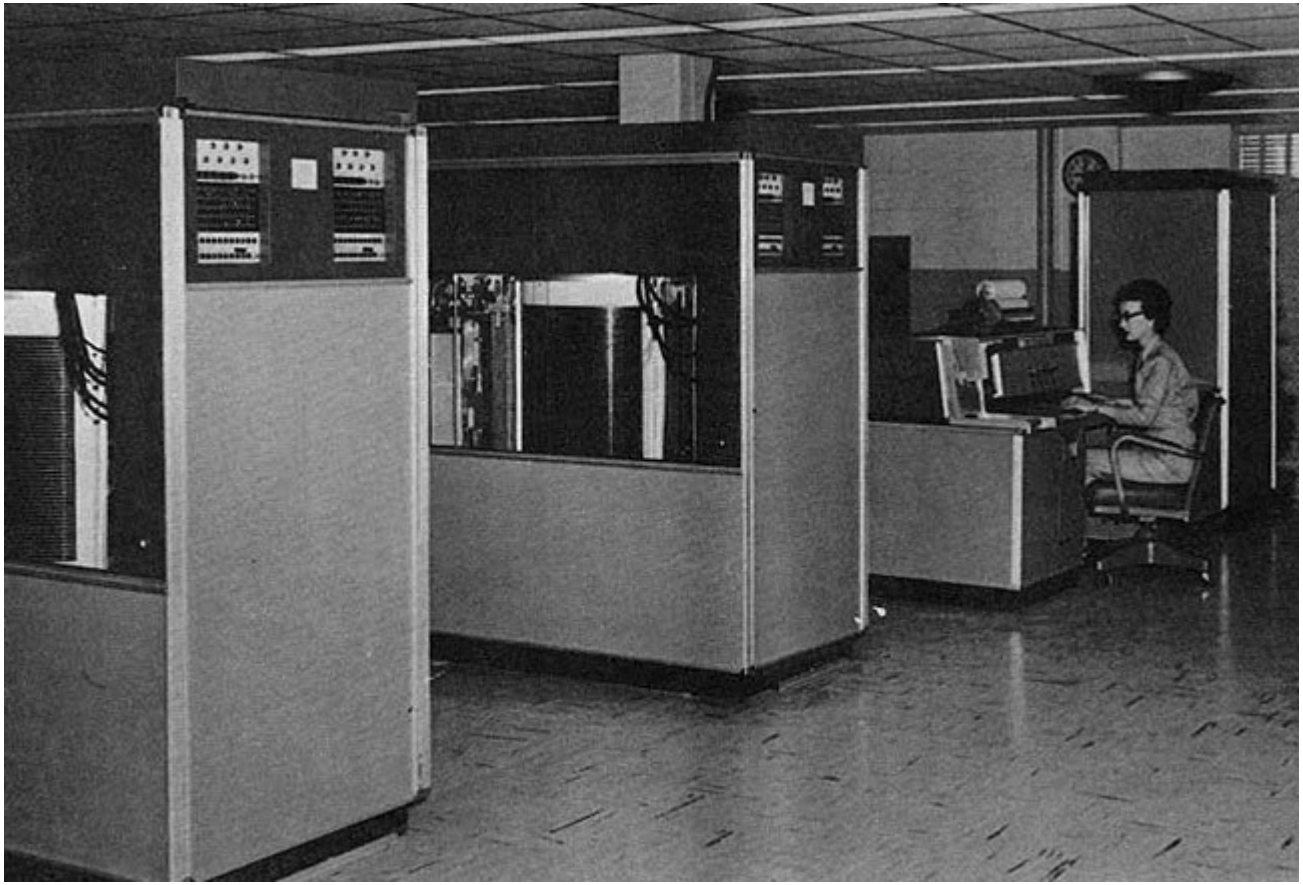
1. Point de vue langages

2ième génération : de 1958 à 1965

- Ere des langages historiques : Fortran IV, Algol, Cobol, APL...

- Macro-assembleurs

IBM sort le Ramac 305
premier ordinateur à disque dur
en 1956

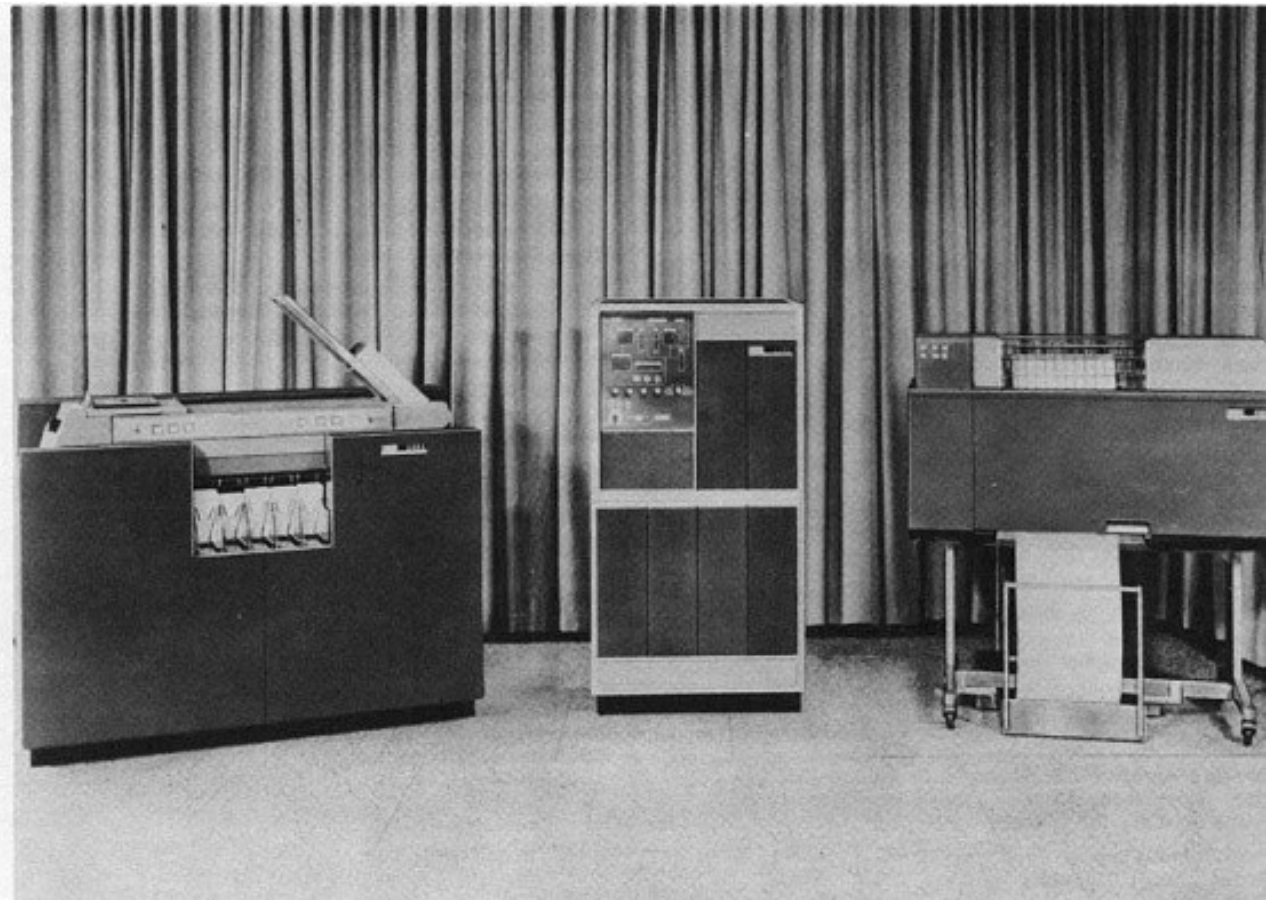


1. Point de vue langages

2ième génération : de 1958 à 1965

- Ere des langages historiques : Fortran IV, Algol, Cobol, APL...

- Macro-assembleurs



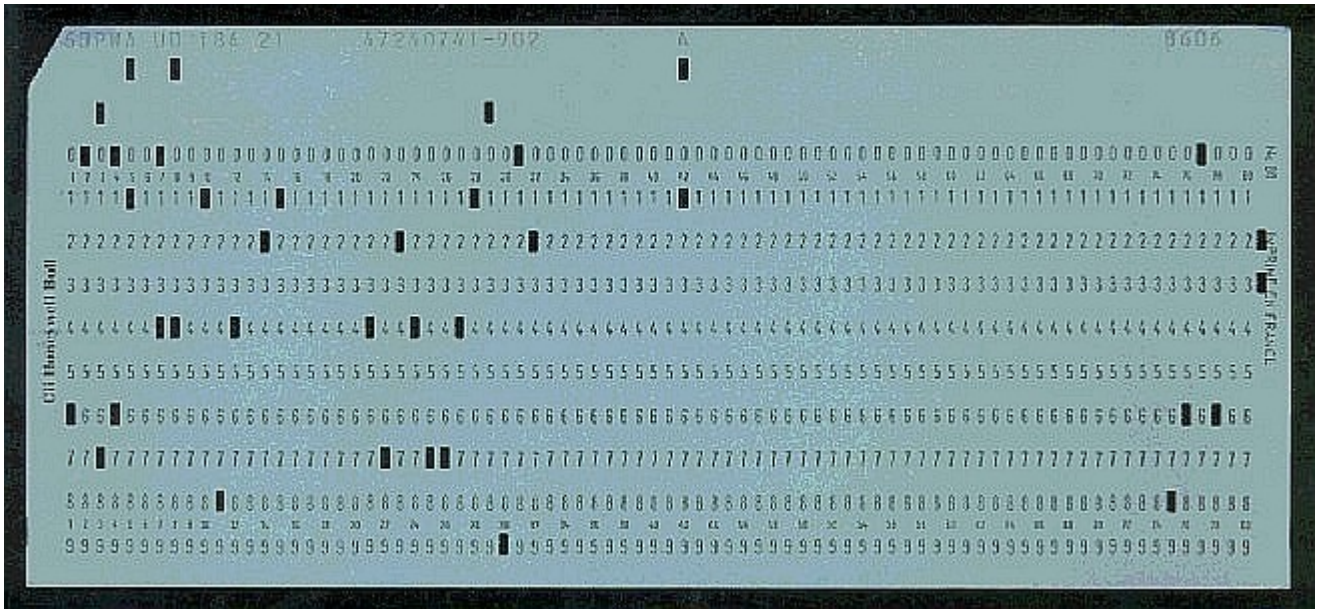
IBM sort l'ordinateur
IBM 1401 à cartes perforées
en 1959

1. Point de vue langages

2ième génération : de 1958 à 1965

- Ere des langages historiques : Fortran IV, Algol, Cobol, APL...

- Macro-assembleurs



Carte perforée pour l'IBM 1401

1. Point de vue langages

3ième génération : de 1966 à 1980 vers l'universalité...

- PL/1 1963
- Basic 1965
- Euler 1966
- Pascal 1970
- **C 1973**...
- ADA 1979 (processus parallèles)



L'IBM PC (Personal Computer) en 1981

...en 1980 il y avait plus de 400 langages recencés ...

1. Point de vue langages

4 ième génération : de 1980 à 2000 Approche Objet

- Prolog, Smalltalk, Simula,
- C++, Eiffel, Java

1. Point de vue langages

4 ième génération : de 1980 à 2000 Approche Objet

- Prolog, Smalltalk, Simula,
- C++, Eiffel, Java

5 ième génération : depuis 2000, approche “réseau” et distribution du code

- html, xhtml
- php
- xml
- Applet Java, ActiveX



2. Point de vue de l'utilisation des ordinateurs

Jusqu'en 1956 :

- absence de logiciel constructeur
- utilisateur charge son programme et surveille l'execution
- mise au point par clef directement sur le matériel
- sur les ordinateurs de 2e génération : traitement par lot

2. Point de vue de l'utilisation des ordinateurs

Jusqu'en 1956 :

- absence de logiciel constructeur
- utilisateur charge son programme et surveille l'execution
- mise au point par clef directement sur le matériel
- sur les ordinateurs de 2e génération : traitement par lot

De 1956 à 1965 : l'homme s'éloigne de la machine...

- premier système d'exploitation (IBM 704) 1956
- premier système de contrôle des E/S (driver) vers 1960
- premier langage de contrôle des travaux (batch) 1962

=> utilisation plus rationnelle de la machine!

2. Point de vue de l'utilisation des ordinateurs

De 1965 à 1990 : Multi-programmation, mono-processeur

- présence en mémoire centrale de plusieurs programmes
- accès par plusieurs utilisateurs
- problème d'attribution des ressources logicielles et matérielles (imprimantes, mémoires, etc)
- priorité d'execution (politique), partage du temps
- ramasse-miette et protection mémoire
- ré-entrance des programmes

2. Point de vue de l'utilisation des ordinateurs

Depuis 1990 :

- Multi-programmation, multi-processeurs

- systèmes parallèles (exemple Intel core Duo)

- systèmes répartis (exemple téléphones portables, réseaux de capteurs)

- Systèmes embarqués, systèmes enfouis

- réactivité très forte vis à vis de l'environnement

- temps réel

- importance de la fiabilité et de la sécurité (exemple ABS)

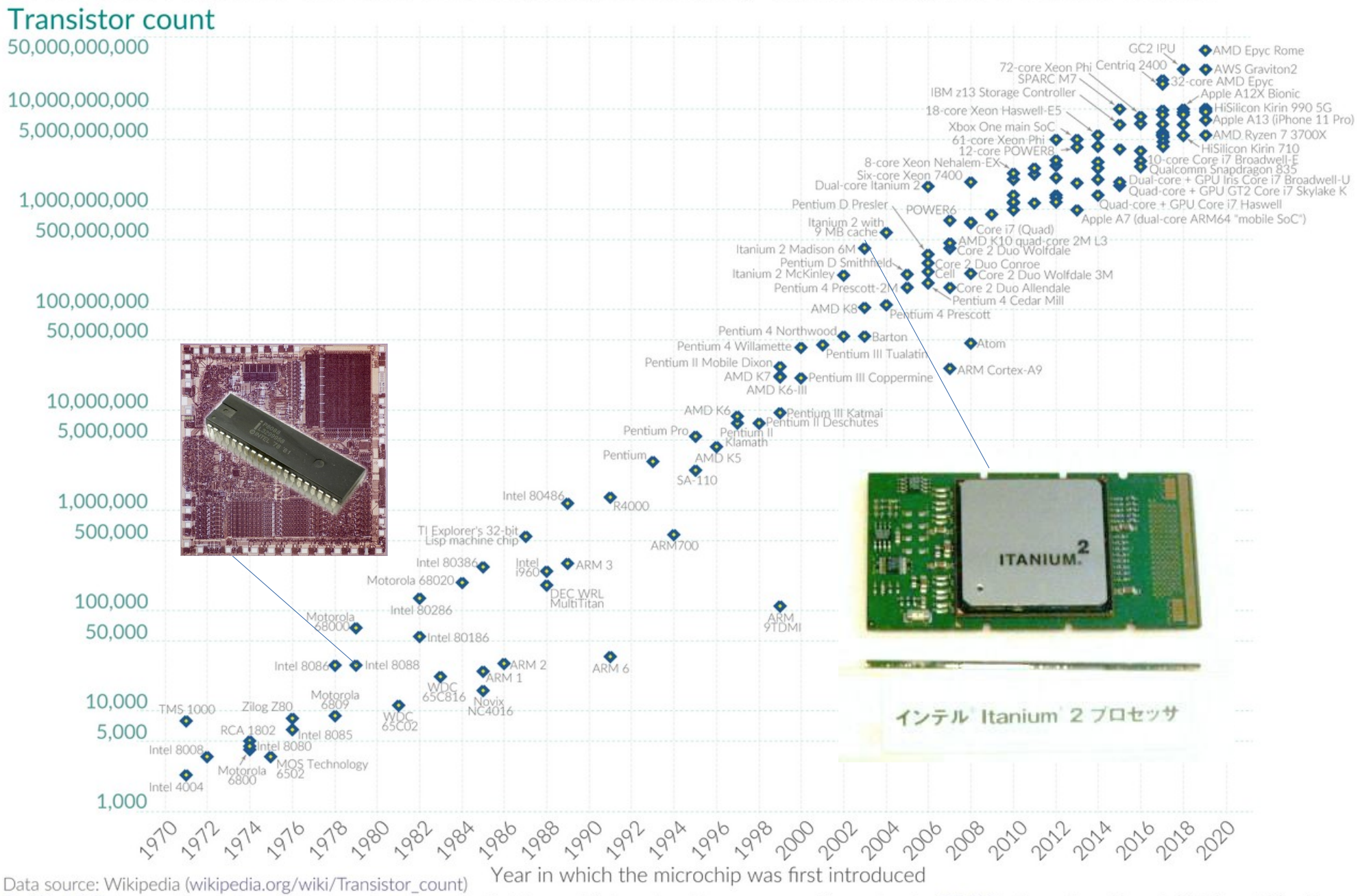
3. Évolution des systèmes informatiques

- Depuis 1970, les micro-processeurs n’ont cessé d’évoluer, en capacités (nombre de transistors), en fréquence (vitesse d’exécution), en finesse de gravure (technologie des transistors) ou en nombre de dispositifs !
- Depuis 1970, la seule loi qui continue à exister est la loi de Moore, qui dit que le nombre de transistors est multiplié par deux tous les deux ans !

Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.



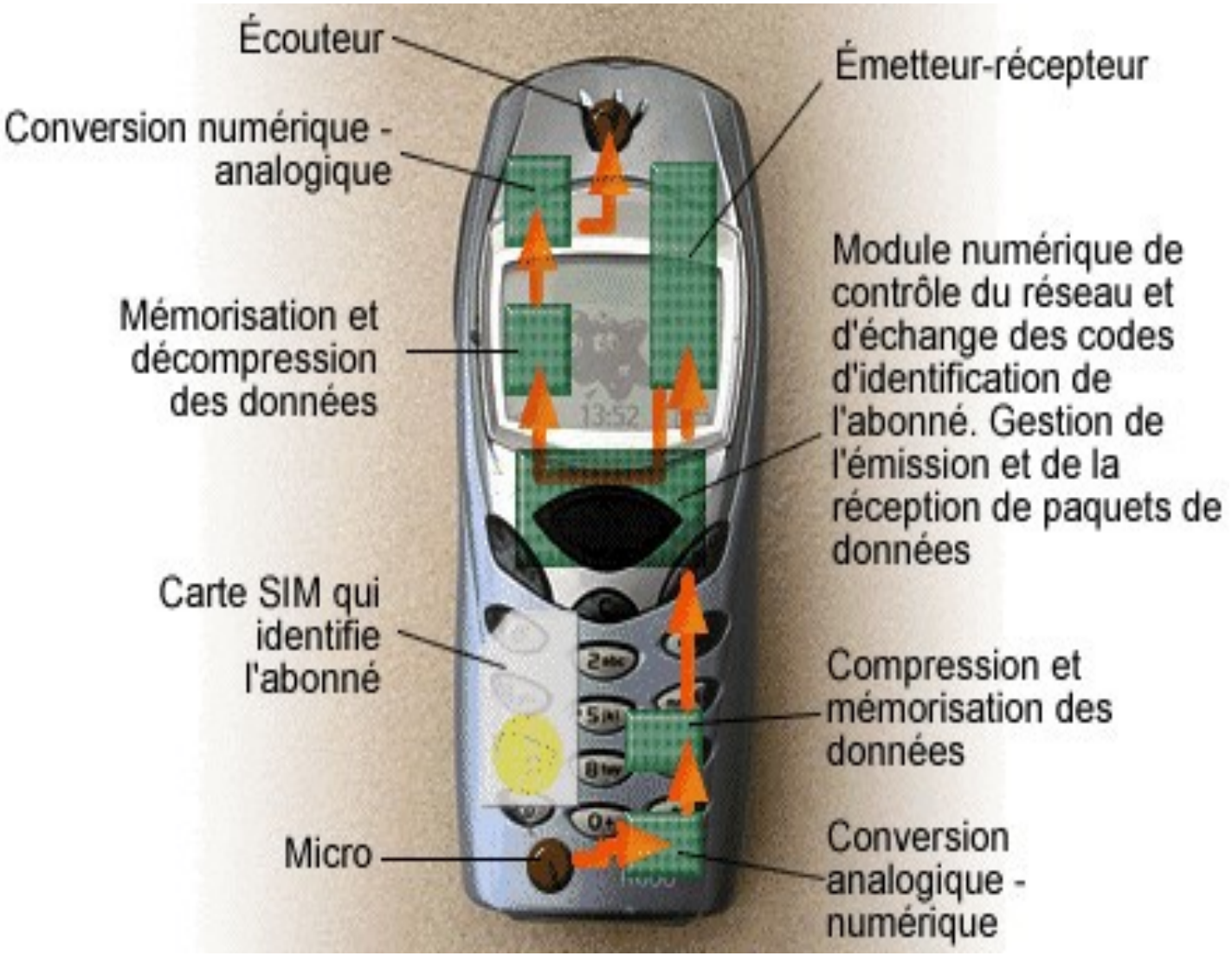
Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Par Max Roser, Hannah Ritchie — https://ourworldindata.org/uploads/2020/11/Transistor-Count-over-time.png, CC BY 4.0, https://commons.wikimedia.org/w/index.php?curid=98219918

Exemple 1 – Téléphone mobile

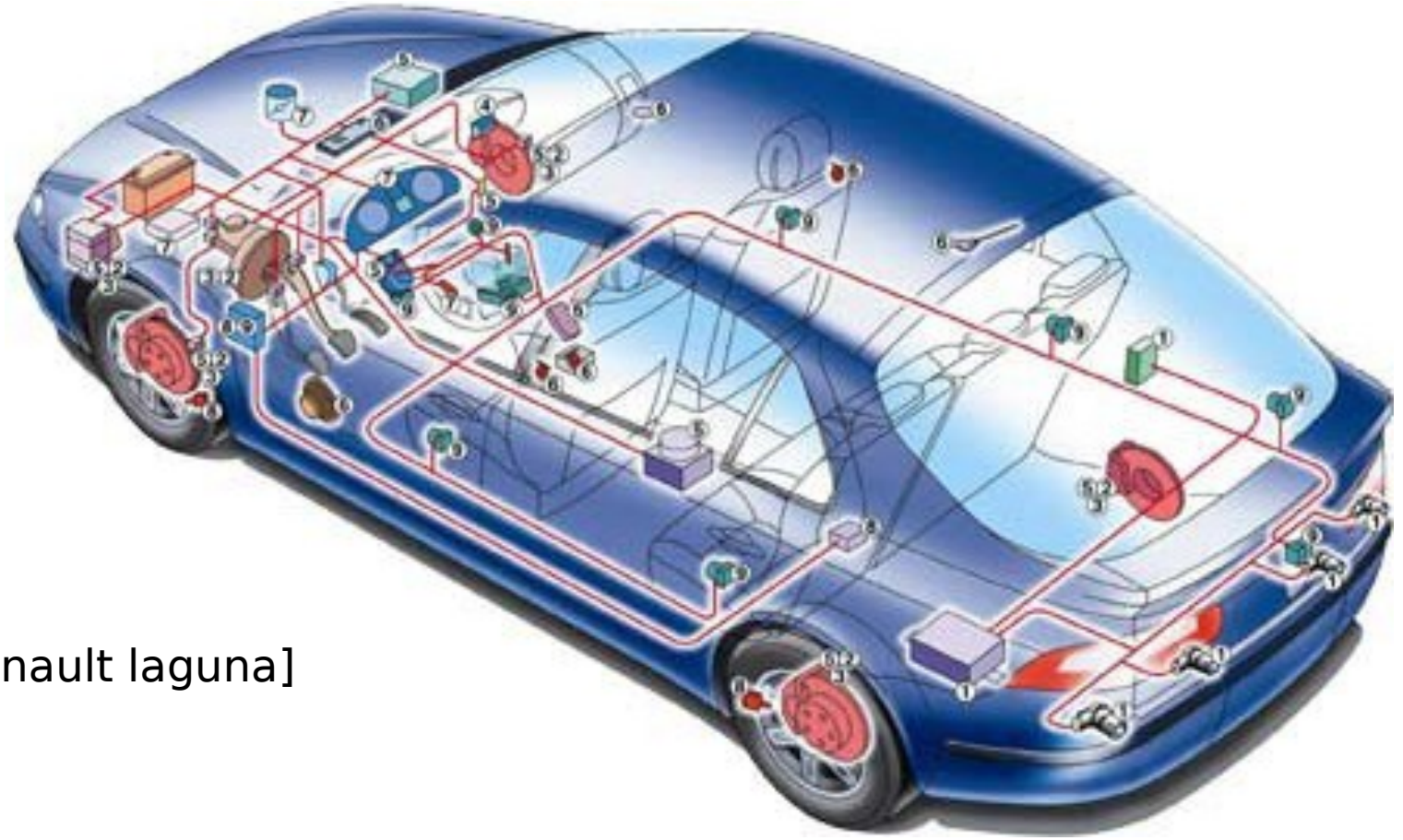


Exemple 2 – Téléphone mobile



Exemple 3 - Automobile

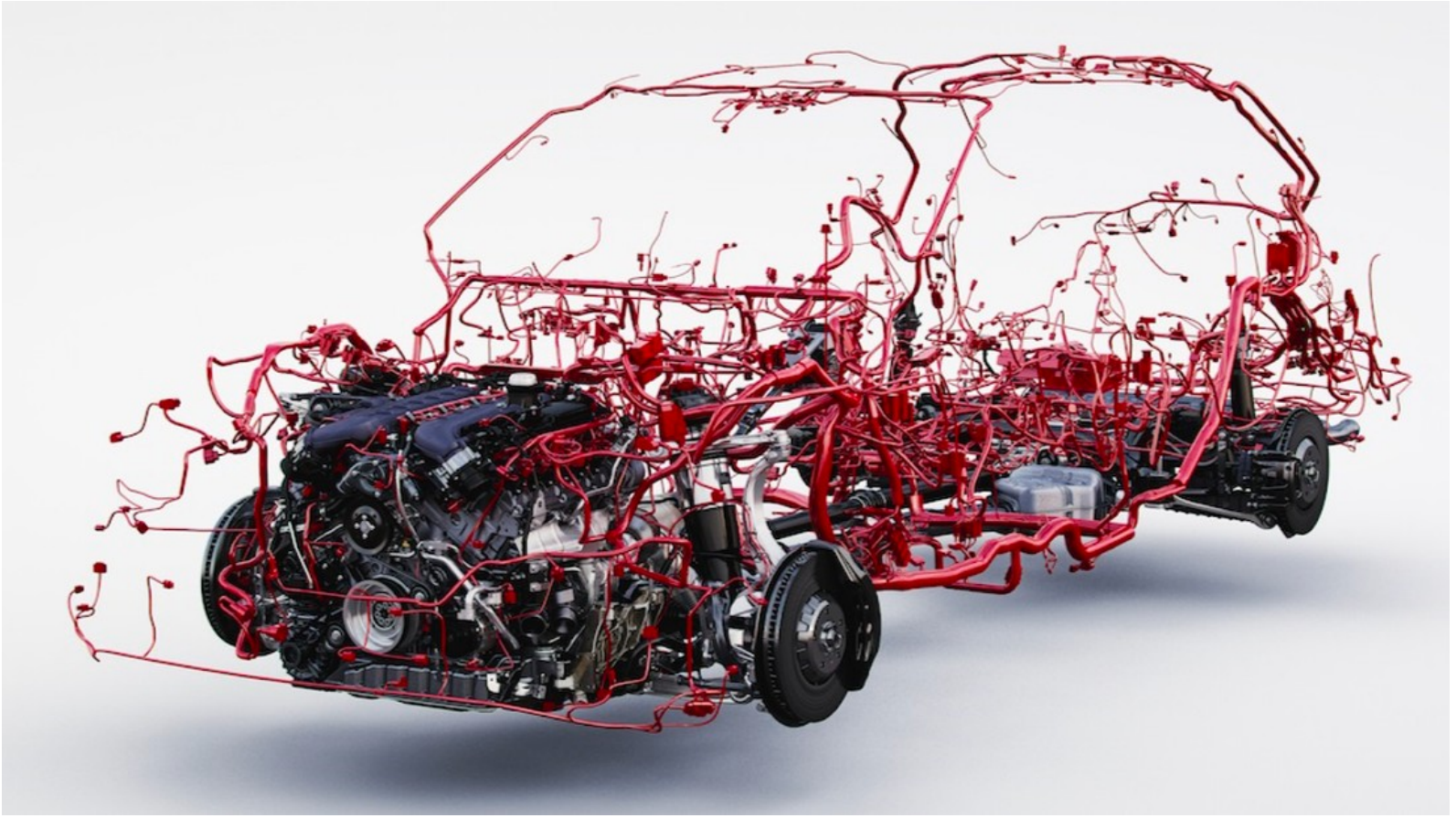
Une automobile de haute classe contient, en moyenne, une **centaine** de microprocesseurs 8 bits à 32 bits...



[Renault laguna]

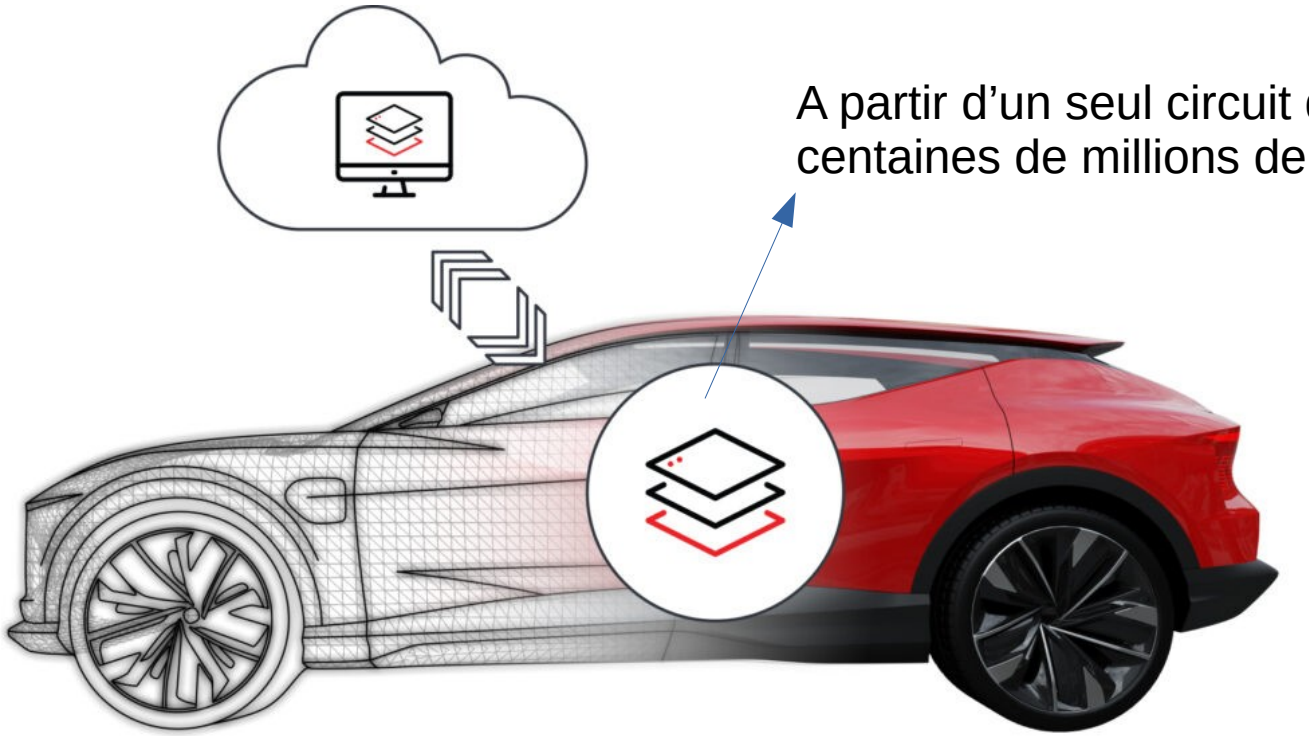
Exemple 4 - Automobile

La quantité de câblage était devenue très prohibitive !



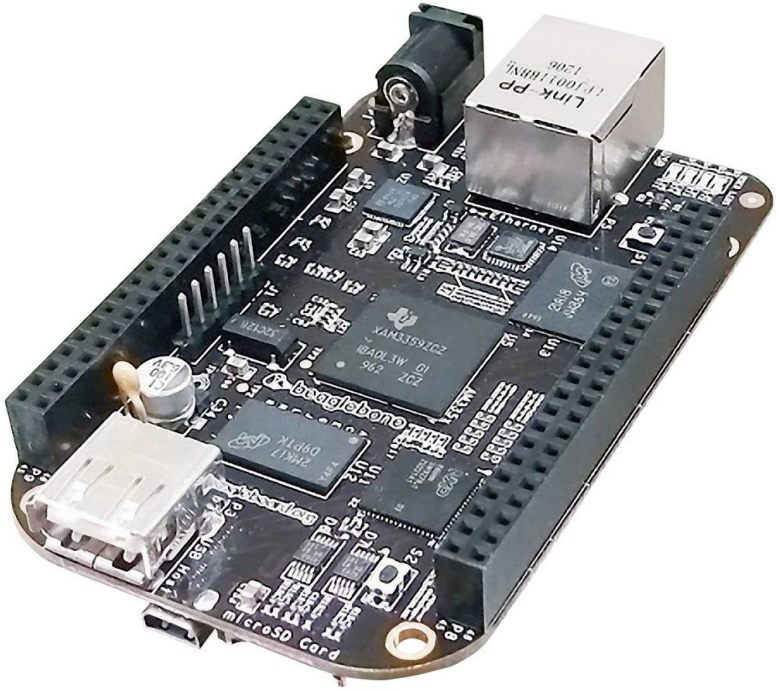
Exemple 5 - Automobile

LA solution : penser à un véhicule comme un ordinateur, qui serait entièrement paramétrable et reconfigurable = le Software Defined Vehicle (SDV)



A partir d'un seul circuit de plusieurs centaines de millions de portes !

Exemple 6 – Cartes à microprocesseurs

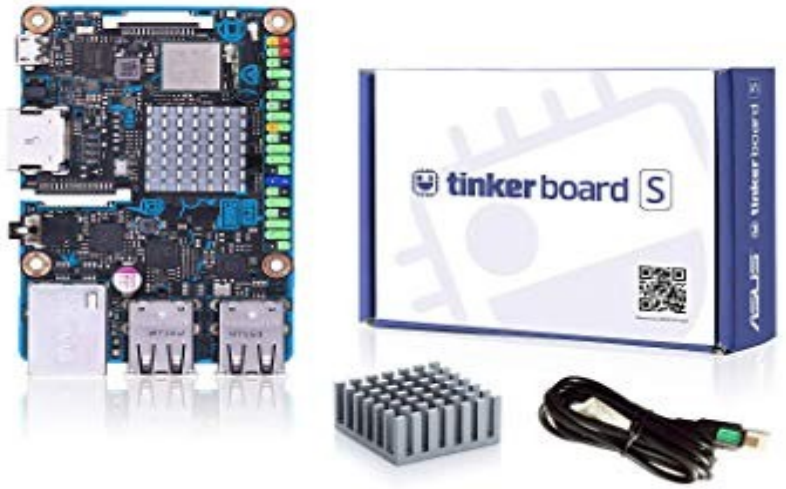


BeagleBoard Black
Linux (Debian, Ubuntu)
Android



Raspberry Pi Model 3 B+
Linux Raspbian

Exemple 6 – Cartes à microprocesseurs

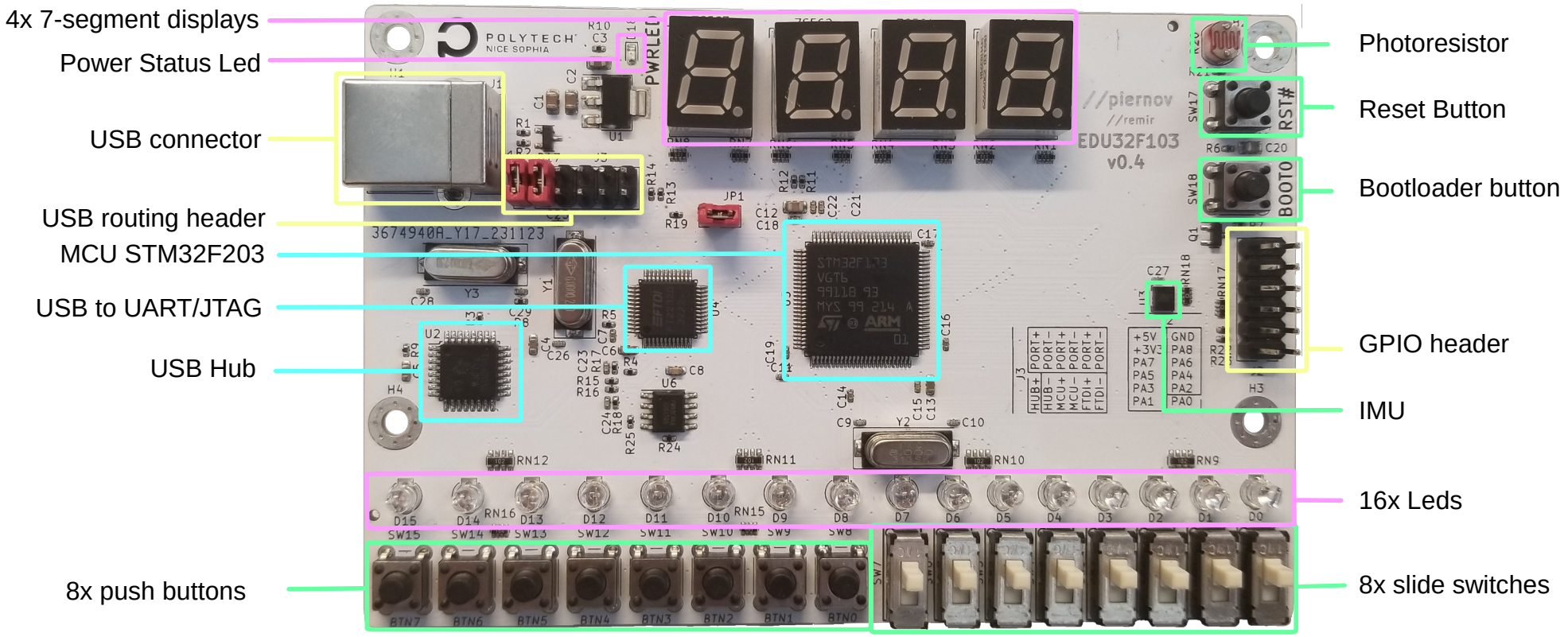


Asus Tinker Board
Linux (Ubuntu)
Android



Raspberry Pi Model 4
Linux

Exemple 6 – Cartes à microprocesseurs

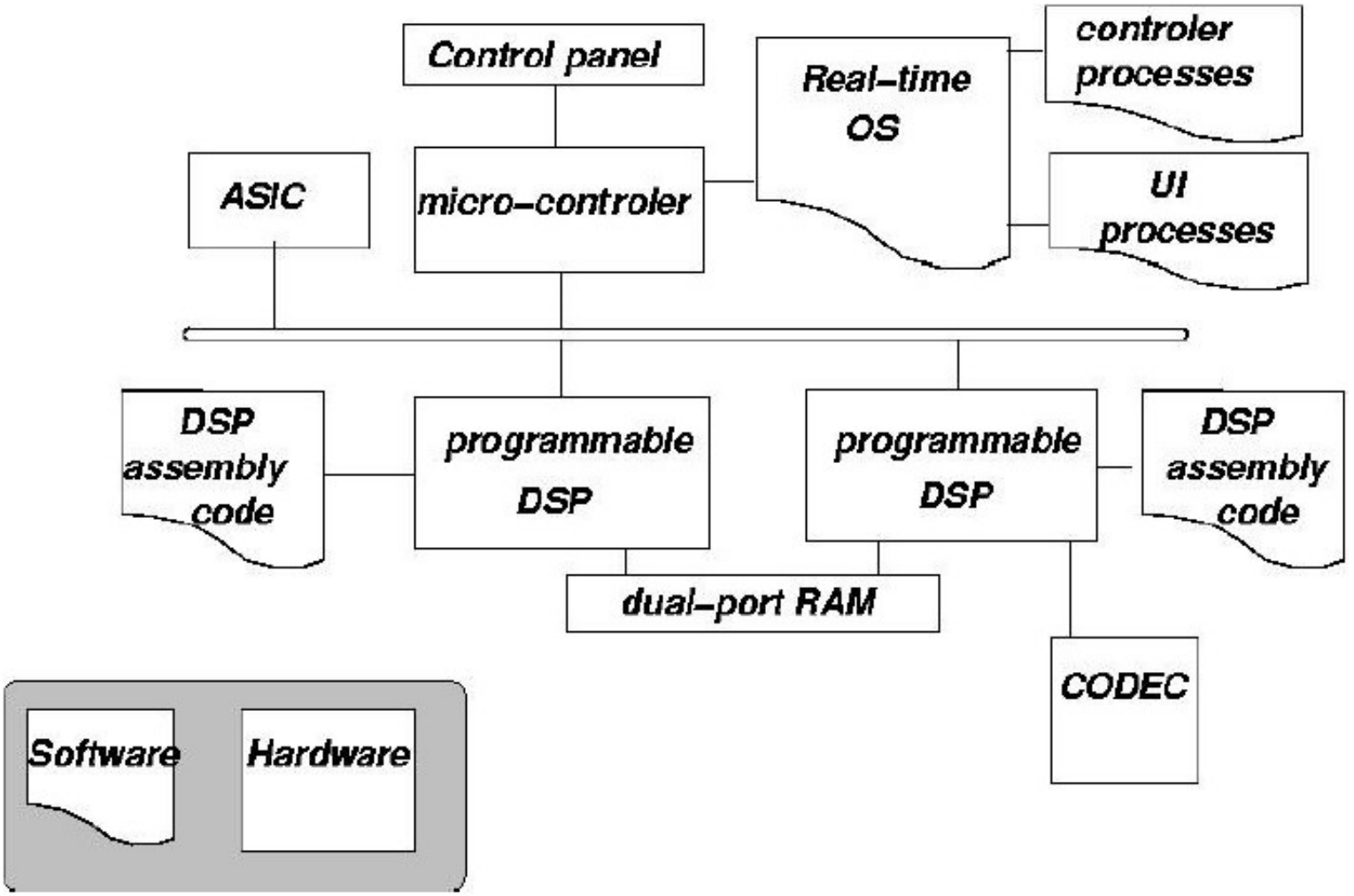


Carte EDU32F103

3. Architectures embarquées

Besoins	Enfoui	Léger	Moyen	Haut de gamme	Haute disponibilité
RAM	< 10 Ko	< 4Mo	2-8 Mo	8-32 Mo	> 100 Mo
ROM/Flash	~ 100 Ko	< 2Mo	2-4 Mo	4-16 Mo	> 100 Mo
Processeurs	PIC NeC Fujitsu Freescale SoC	DragonBall 68k ColdFire ARM DSP	MIPS ColdFire ARM PowerPC	MIPS ARM PowerPC x86	PowerPC X86
Matériel sup	Pas de cache	Pas de MMU	SoC	Video controller	≅ PC
Exemple	u-contrôleur (automobile, avionique...)	PDA, tel, machine à laver	Routeur, Wifi, capteurs	Console	Serveur

3. Architecture représentative d'un système embarqué



3. Rôle d'un système d'exploitation

En complexité croissante :

- chargeur : initialisation du système (cf BIOS dans les PC)
- gestion des E/S
- multi-traitements
- gestion du temps (Time Sharing)
- gestion de la mémoire

3. Rôle d'un système d'exploitation

Fonctions d'un système d'exploitation :

- gestion des travaux
- aides au traitement
- traitement des erreurs
- gestion des données
- utilitaires d'exploitation
- maintenance des programmes (gestion des bibliothèques)



1.2 Différences entre un système d'exploitation et un RTOS

Les systèmes temps-réel

Définition

Ce sont des systèmes liés au contrôle de procédés du monde (temps) **réel**.

L'exécution de traitements dans ces systèmes doit se terminer avant une date butoir, appelée ***échéance***, définie par la vitesse d'évolution de l'environnement **réel**, au-delà de laquelle les résultats ne sont plus valides.

Exemple critique : le contrôleur de frein d'une voiture, la sortie du train d'atterrissage d'un avion…

Les systèmes temps-réel

- **Système temps-réel dur/strict** (Hard Real-Time)

- **Système temps-réel mou** (Soft Real-Time)

Les systèmes temps-réel

- **Système temps-réel dur/strict** (Hard Real-Time)

Le non-respect des échéances peut avoir des conséquences graves sur le fonctionnement du système ou sur son environnement (auto-pilotage, freinage, assistance médicalisée…).

Les échéances ne doivent **jamais** être dépassées.

- **Système temps-réel mou** (Soft Real-Time)

Les systèmes temps-réel

- **Système temps-réel dur/strict** (Hard Real-Time)

Le non-respect des échéances peut avoir des conséquences graves sur le fonctionnement du système ou sur son environnement (auto-pilotage, freinage, assistance médicalisée…).

Les échéances ne doivent **jamais** être dépassées.

- **Système temps-réel mou** (Soft Real-Time)

Le non-respect des échéances ralentit le système sans conséquences graves (billetterie automatique…).

Le système doit répondre au mieux, le plus rapidement possible → notion de QoS (qualité de service).

Les systèmes temps-réel

- Exemples de systèmes **temps-réel dur** :
  - Pilotage automatique de la ligne 14 du métro
  - Dispositif de surveillance d'une centrale nucléaire
  - Système de guidage de missiles
  - Régulateur de vitesse d'une automobile

Les systèmes temps-réel

- Exemples de systèmes **temps-réel dur** :
  - Pilotage automatique de la ligne 14 du métro
  - Dispositif de surveillance d'une centrale nucléaire
  - Système de guidage de missiles
  - Régulateur de vitesse d'une automobile
- Exemples **Temps-réel mou** :
  - Un téléphone portable (ex. Webphone)
  - Lecteur DVD, électroménager...

Informatique :

- Processeur standard
  - Multiples unités fonctionnelles (flottant, exécution désynchronisée, prédiction…)
  - Fréquence élevée (> GHz)
  - Consommation électrique élevée
  - Chaleur
  - Taille
- MMU (mémoire virtuelle)
- OS
- Cache
- Grand nombre de périphériques

Embarqué :

- Processeur dédié (contrôleur)
  - Architecture adaptée
  - Vitesse faible (~200 MHz)
  - 8-32bits : mémoire limitée
  - Basse consommation
  - Petite taille, grand volume => faible coût
- Processeur DSP (traitements)
  - Très puissants
- Qqs Mo de mémoire
- RTOS
- Pas de cache

Les systèmes Temps Réel

- Il ne s'agit pas de rendre le résultat le plus rapidement possible, mais simplement **à temps**.
- L'échelle de temps de l'échéance peut varier d'une application à une autre :
  - Microsecondes en contrôle radar,
  - Millisecondes pour la synchronisation image/son (MPEG)
  - Minutes pour les distributeurs automatiques
- Tous peuvent être temps-réel souples ou durs



1.3 Toute petite bibliographie sur les systèmes temps réels

« Systèmes temps réel 1 & 2 »
De Nicolas Navet, Hermes Lavoisier



« Real-Time Systems »
De Jane W. S. Liu, Prentice Hall

